

William Stallings
Computer Organization
and Architecture
10th Edition

Chapter 16
Instruction Level Parallelism
and Superscalar Processors

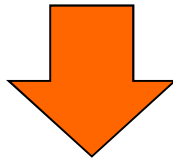
Key points

Instructions
vary in their
duration

A floating-point **addition** or **multiplication** needs **1-3** clock cycles. @intel Pentium 4

These instructions
can often be
executed in parallel

A **division** takes **39** clock cycles
@intel Pentium 4



Superscalar processor

Key points

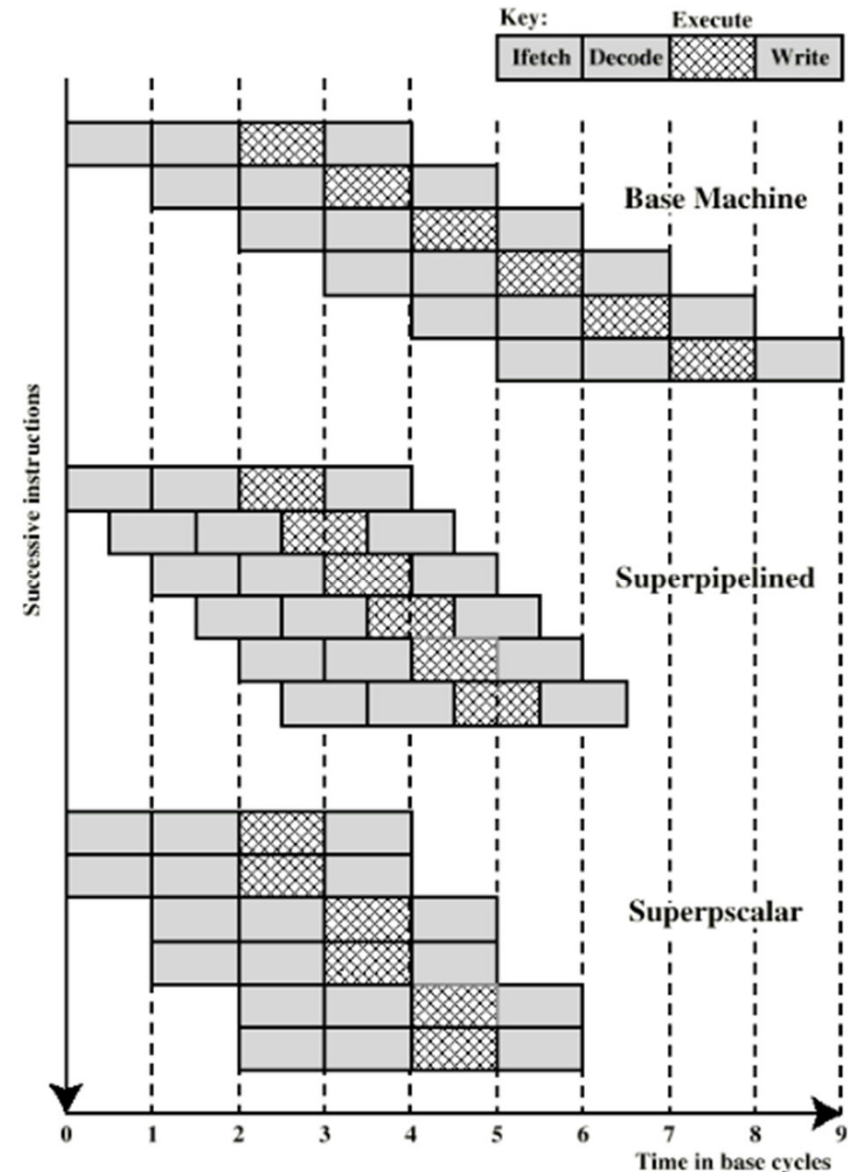
- Superscalar is to use multiple *independent* instruction pipelines in a processor exploring instruction-level parallelism.
- A superscalar processor fetches multiple instructions at a time and attempts to find nearby instructions that are *independent* of one another.

Key points

- The processor may *eliminate* some *unnecessary dependencies* by the use of additional registers and the renaming of register references in the original code.
- Most superscalar machines use **branch prediction** methods *rather than delayed branches* to improve efficiency.

What is Superscalar?

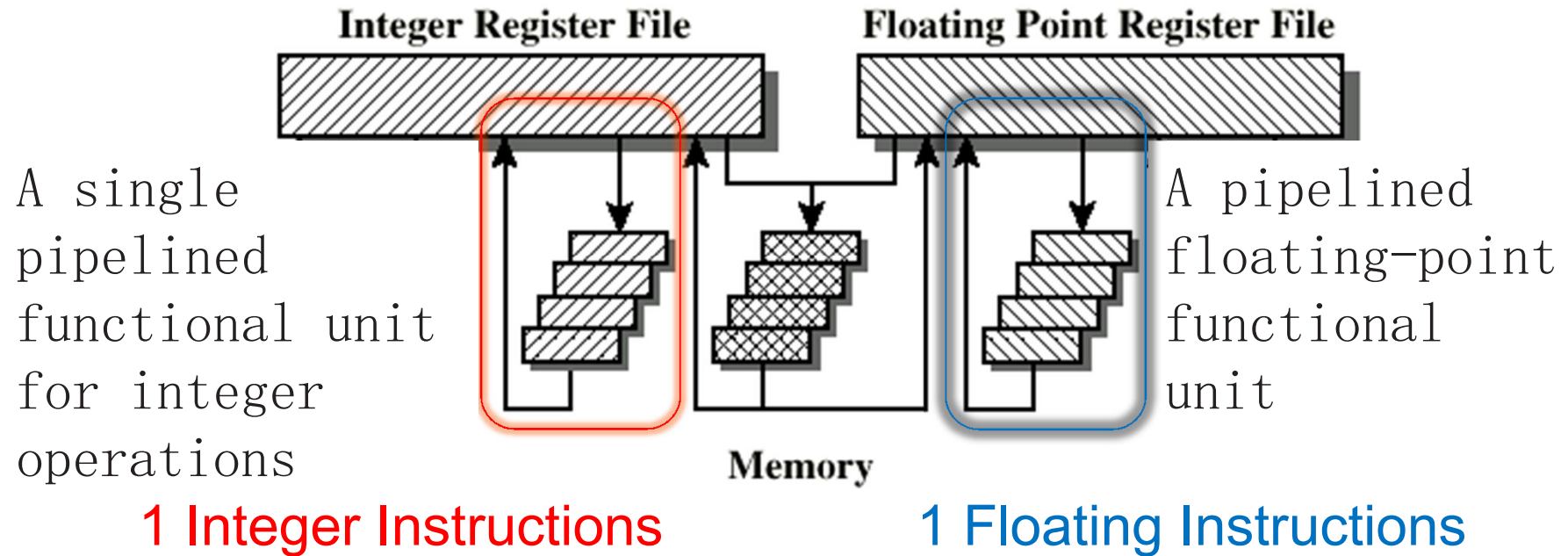
- Common instructions (arithmetic, load/store, conditional branch) can be initiated *simultaneously* and executed *independently*
- Equally applicable to *RISC & CISC*
- Standard method for high performance microprocessors



16.1 Overview

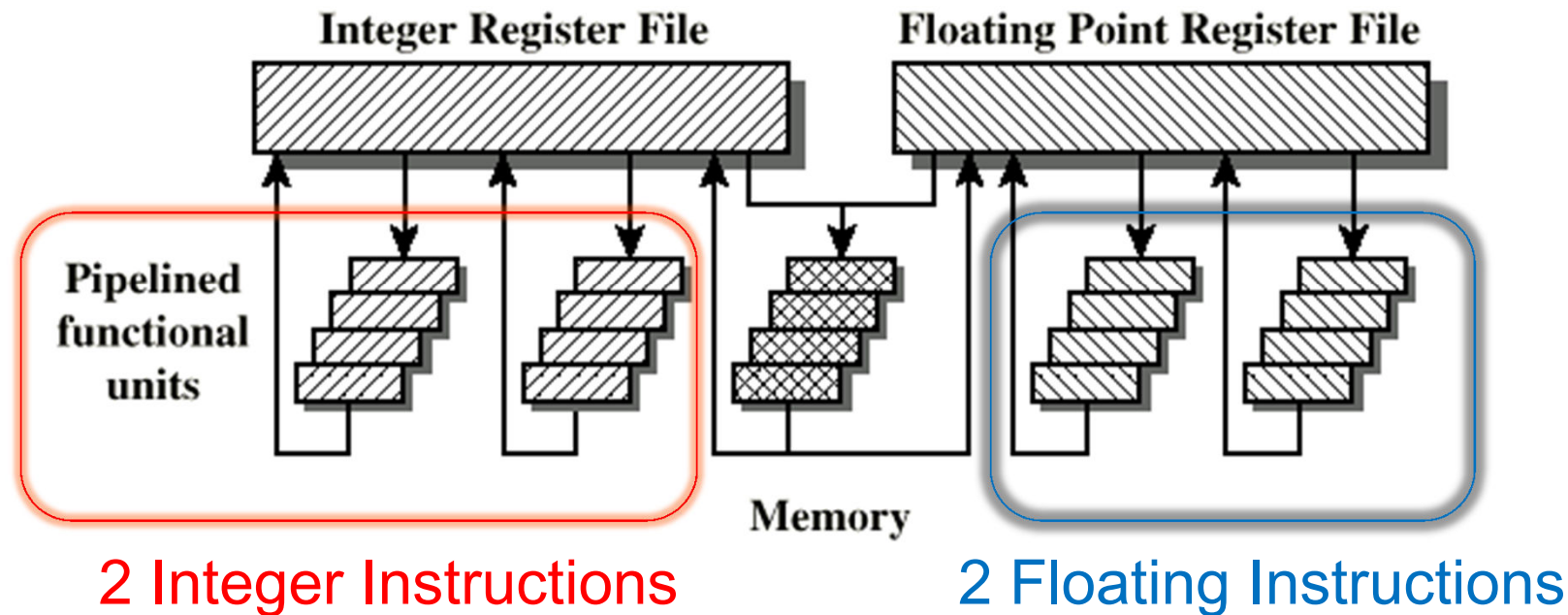
- The term superscalar first coined in 1987, refers to a machine that is designed to improve the performance of the execution of scale instructions
- The **essence** of the superscalar approach is the ability to execute instructions **independently** and **concurrently** in different pipelines.
- The concept can be further exploited by allowing instructions to be executed in an order **different** from the program order.

General Superscalar Organization



Scalar: Enable multiple **instructions** to be at different pipeline **stages** at one time.

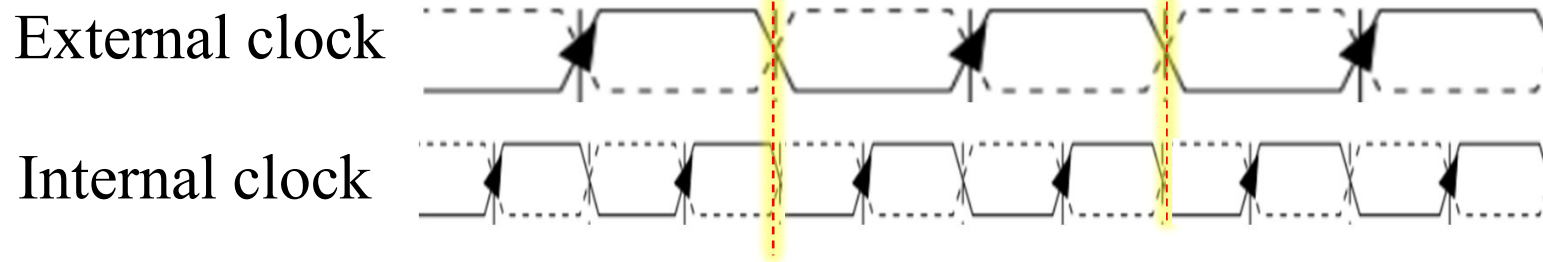
General Superscalar Organization



Superscalar: there are multiple functional units, each of which is implemented as a pipeline. Each individual functional unit provides a degree of parallelism.

Superscalar VS Superpipeline

- Superpipeline
 - ✓ Many pipeline stages need *less than half a clock cycle*
 - ✓ **Double** internal clock speed gets two tasks per external clock cycle



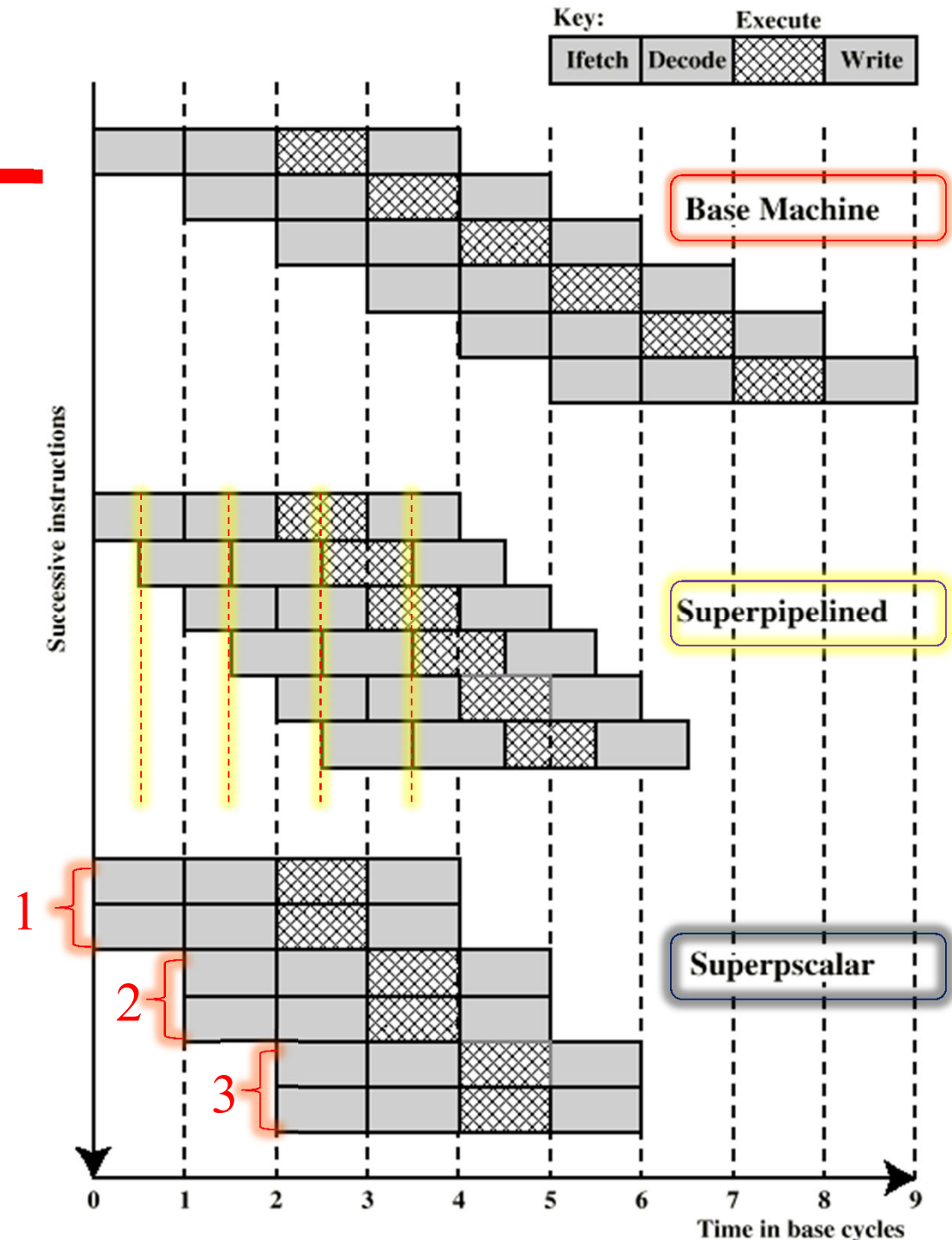
- Superscalar allows **parallel** fetch execute

Superscalar vs Superpipeline

Table 16.1 Reported Speedups of Superscalar-Like Machines

Reference	Speedup
[TJAD70]	1.8
[KUCK77]	8
[WEIS84]	1.58
[ACOS86]	2.7
[SOHI90]	1.8
[SMIT89]	2.3
[JOUP89b]	2.2
[LEE91]	7

The superpipelined processor **falls behind** the superscalar processor at the start of the program and at each branch target.



Limitations

- Instruction level parallelism(**ILP**): the degree to which, on average, the instructions of a program can be executed in parallel.
- How to improve **ILP**?
 - ✓ *Compiler* based optimization
 - ✓ *Hardware* techniques
- Limited by
 - True data *dependency*
 - Procedural *dependency*
 - Resource *conflicts*
 - Output *dependency*
 - Antidependency

True Data Dependency

- ADD EAX, ECX ; EAX = EAX+ECX
 - MOVE EBX,EAX ; EBX = EAX
 - (for Superscalar) Can fetch and decode second instruction in parallel with first
 - Can NOT execute second instruction until first is finished
- ***Flow dependency*** or ***read-after-write dependency***

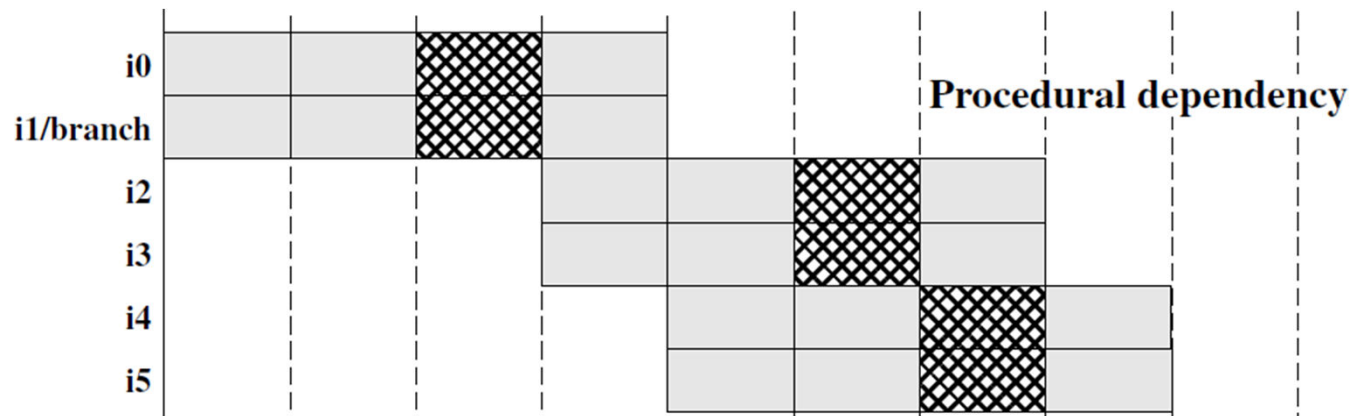
True Data Dependency

- *Memory execution* makes the delay longer
 - Memory access may take 2 or more clock cycles to LOAD/STORE;
 - `MOVE EAX, eff` (load register EAX with the contents of effective memory address eff)
 - `MOV EBX, EAX`
- ✓ For RISC, *reorder instructions* can speed up pipeline
- X For Superscalar, reorder is *less effective* than in RISC pipeline



Procedural Dependency

- ① For **conditional branch**: Can not execute instructions after a **branch** in parallel with instructions before a branch

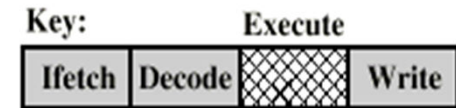


- ② if **instruction length is not fixed** (*i.e. for CISC*), instructions have to be decoded to find out how many fetches are needed.
 - This prevents simultaneous fetches
 - One reason of superscalar being more readily **applicable to RISC**

Resource Conflict

- Two or more instructions requiring access to the ***same resource*** at the same time
 - e.g. two arithmetic instructions, both need ALU device;
- Can duplicate resources
 - e.g. have two arithmetic units

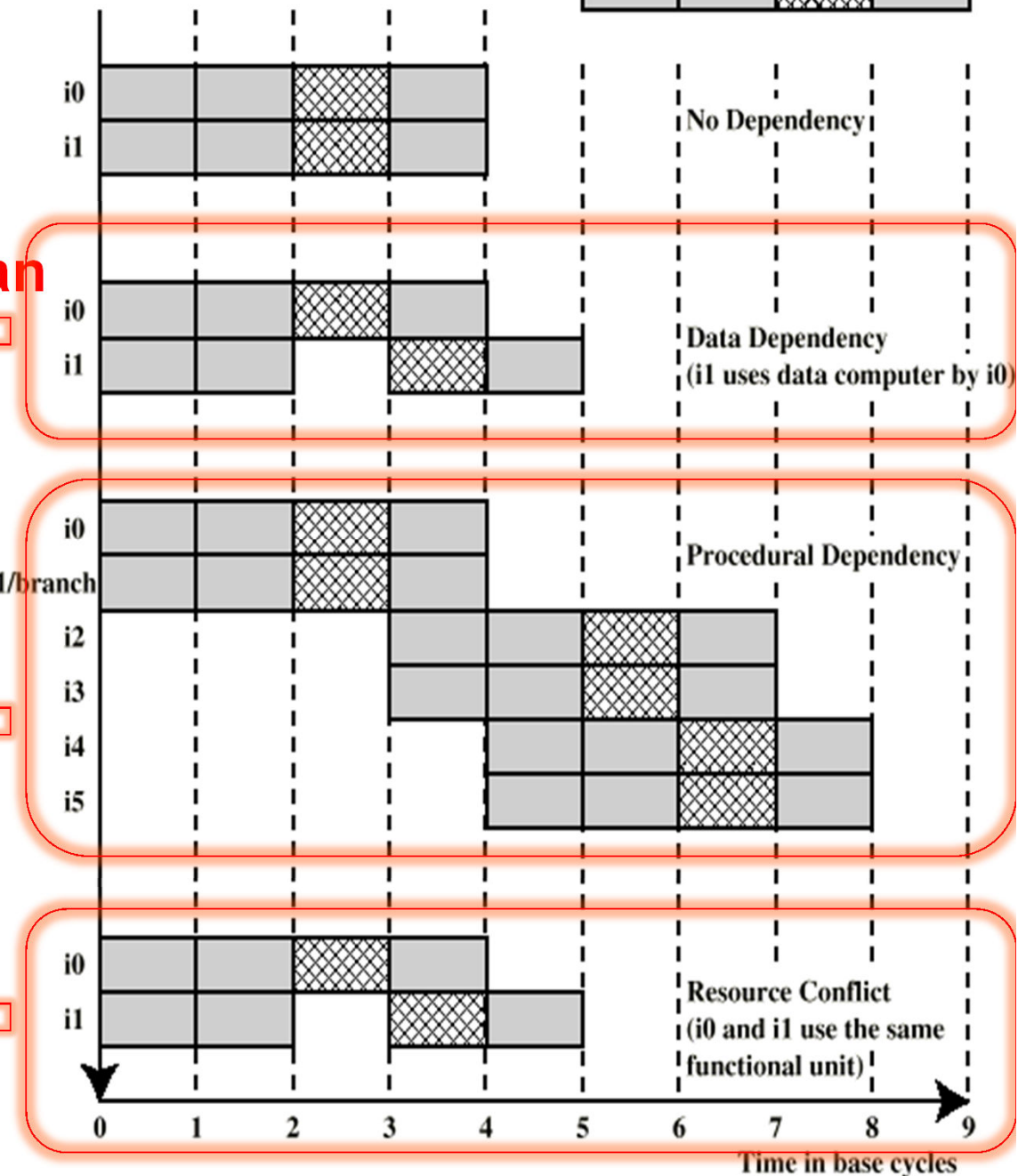
Effect of Dependencies



Typical RISC is better than Superscalar! Reorder is effective.

RISC is more readily than CISC !

Duplicate resource!



Output dependency & Antidependency

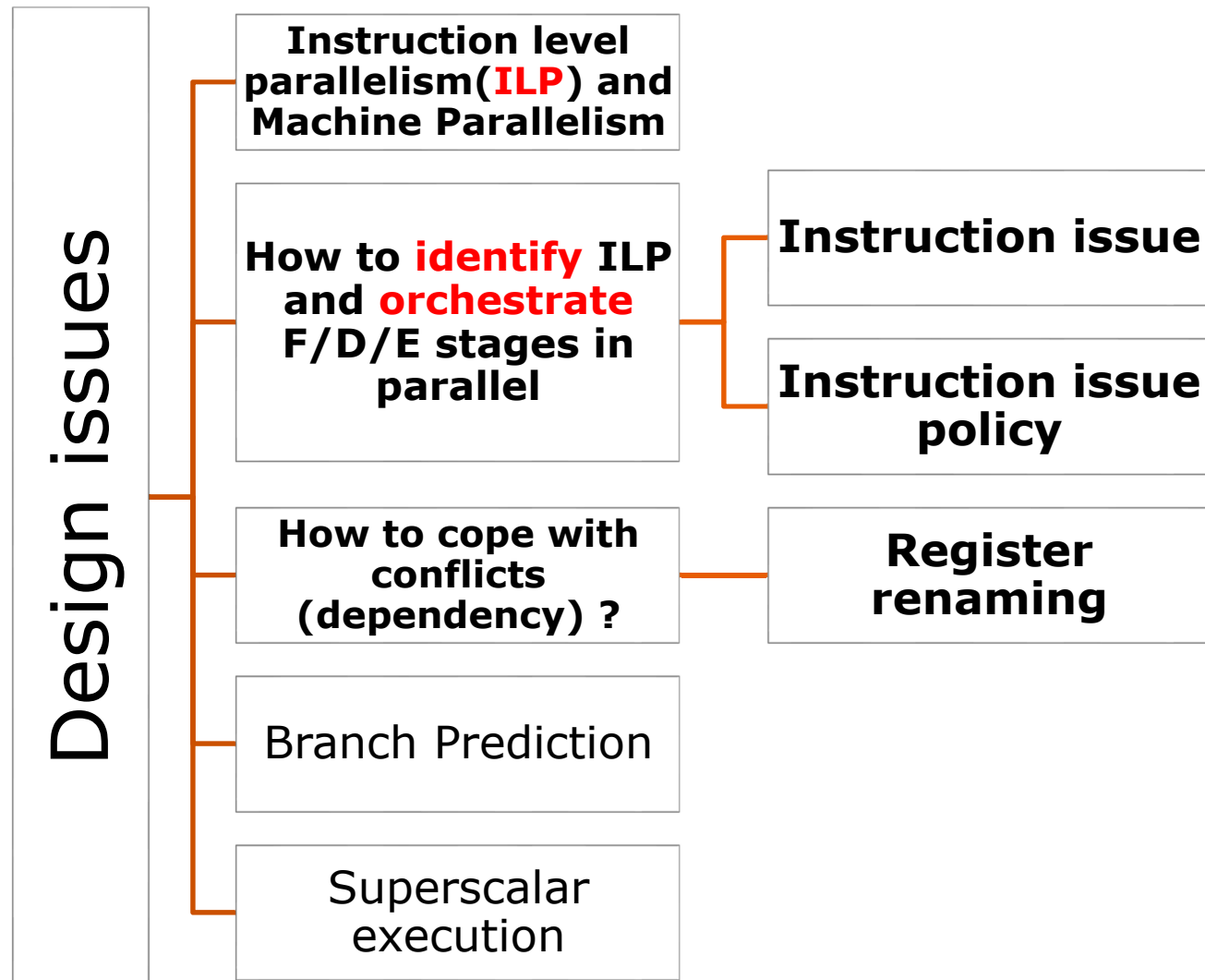
- **Output dependency**

- I1: $R3 \leftarrow R3 \text{ op } R5$
- I2: $R4 \leftarrow R3 + 1$
- I3: $R3 \leftarrow R5 + 1$
- I4: $R7 \leftarrow R3 \text{ op } R4$
- I2 depends on result of I1 - data dependency
- I4 depends on result of I3 - data dependency
- If I3 completes before I1, the result from I1 will be wrong - output (**write-after-write**) dependency

- ***Write-after-Read* dependency (Antidependency)**

- I1: $R3 \leftarrow R3 \text{ op } R5$
- I2: $R4 \leftarrow R3 + 1$
- I3: $R3 \leftarrow R5 + 1$
- I4: $R7 \leftarrow R3 \text{ op } R4$
- I3 can not complete before I2 starts as I2 needs a value in R3 and I3 changes R3

16.2 Design Issues



Instruction level parallelism

- Instructions in a sequence are ***independent*** (key)

```
Load R1 ← R2  
Add R3 ← R3, "1"  
Add R4 ← R4, R2
```

- Execution can be ***overlapped***
- The degree of instruction-level parallelism is determined by the frequency of **true data dependencies** and **procedural dependencies** in the code.
- Also governed by **operation latency**: the time until the result of an instruction is available for use as an operand in a subsequent instruction.

Instruction level parallelism

- E.g1 *Independent* instructions

Load R1 $\leftarrow$ R2

Add R3 $\leftarrow$ R3, "1"

Add R4 $\leftarrow$ R4, R2

- E.g2 *Dependent* instructions (?)

Add R3 $\leftarrow$ R3, "1"

Add R4 $\leftarrow$ R3, R2

Store [R4] $\leftarrow$ R0

Machine Parallelism

- Machine (CPU's) ***ability*** to take advantage of instruction level parallelism
- limited by
 - the ***speed*** and **sophistication** of the mechanisms to find independent instructions
 - number*** of parallel pipelines to be fetched and executed

Instruction Issue Policy

- *Instruction issue*: process of initiation instruction execution in the processor's functional units
- *Instruction issue policy*: protocol used to issue instructions
- CPU must look ahead.
- **Important orders:**
 - Order in which instructions are *fetches*
 - Order in which instructions are *executed*
 - Order in which instructions *change registers and memory*

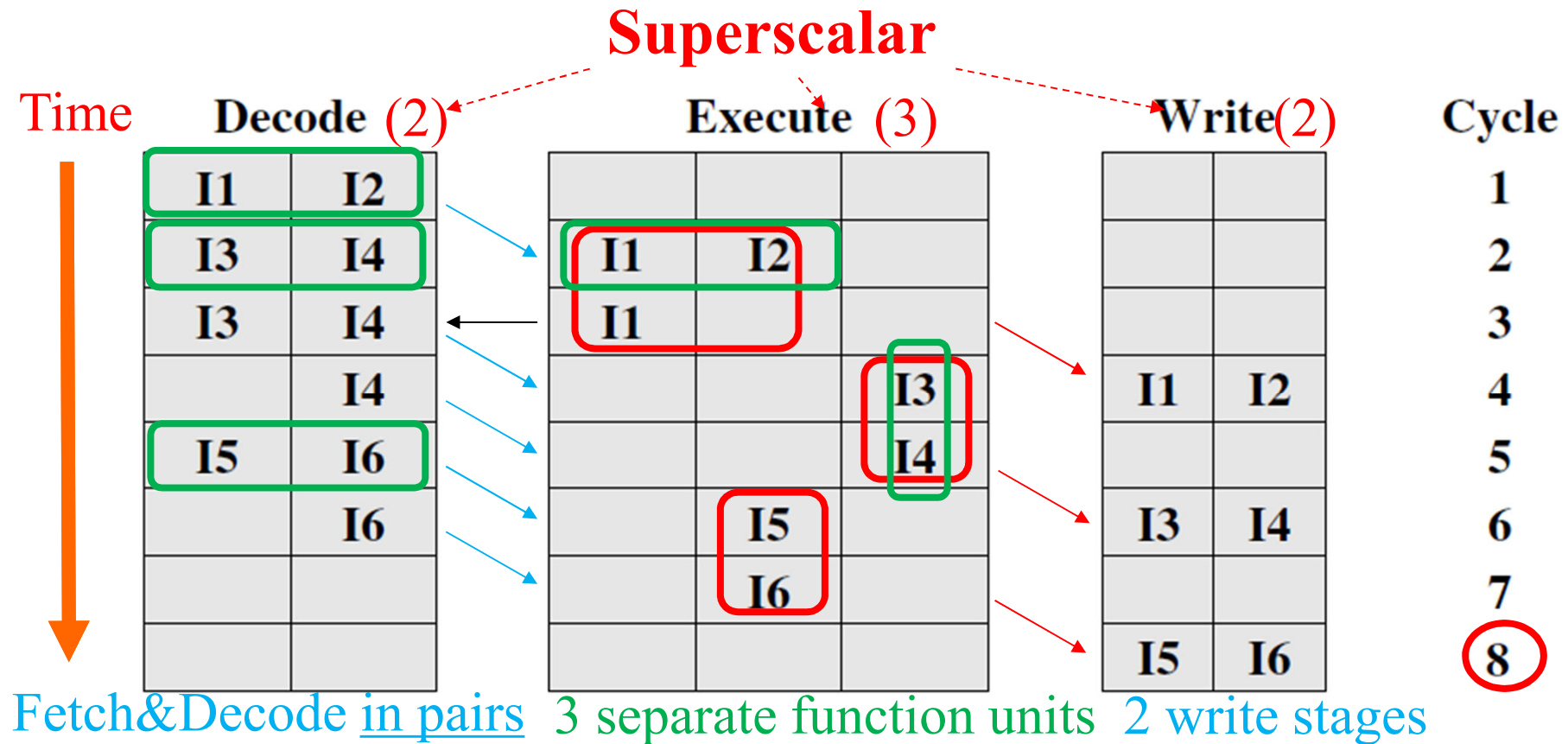
In-Order Issue

In-Order Completion

- Issue instructions in the exact order that would be achieved by **sequential execution** (in-order issue)
 - **Write results** in that same order (in-order completion)
- Not very efficient
- May fetch >1 instruction
- Instructions must stall if necessary (dependency, branch...)

In-Order Issue

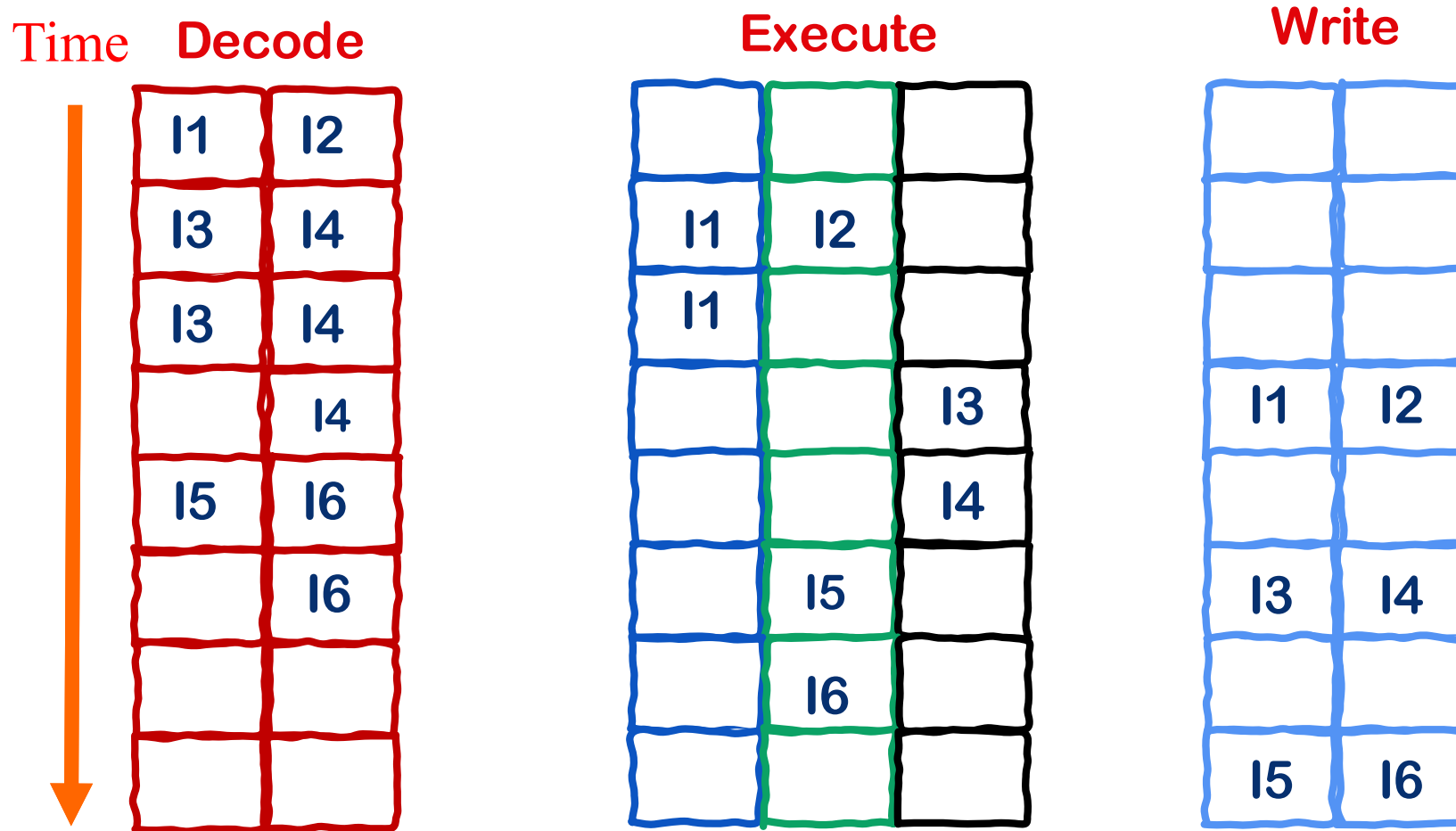
In-Order Completion (Diagram)



- I1 needs 2 time cycles;-----2 time cycles
- I3, I4 need the same function unit; -----resource conflict
- I5 need the outputs of I4;-----true data dependency
- I5, I6 need the same function unit; -----resource conflict

In-Order Issue

In-Order Completion (Diagram)



In-Order Issue

In-Order Completion (Diagram)

	1	2	3	4	5	6	7	8 cycle
I1	D	E	E	W				
I2	D	E		W				
I3		D	D	E		W		
I4		D	D	D	E	W		
I5					D	E		W
I6					D	D	E	W

I1 : 2 E

I3, I4 : resource conflict

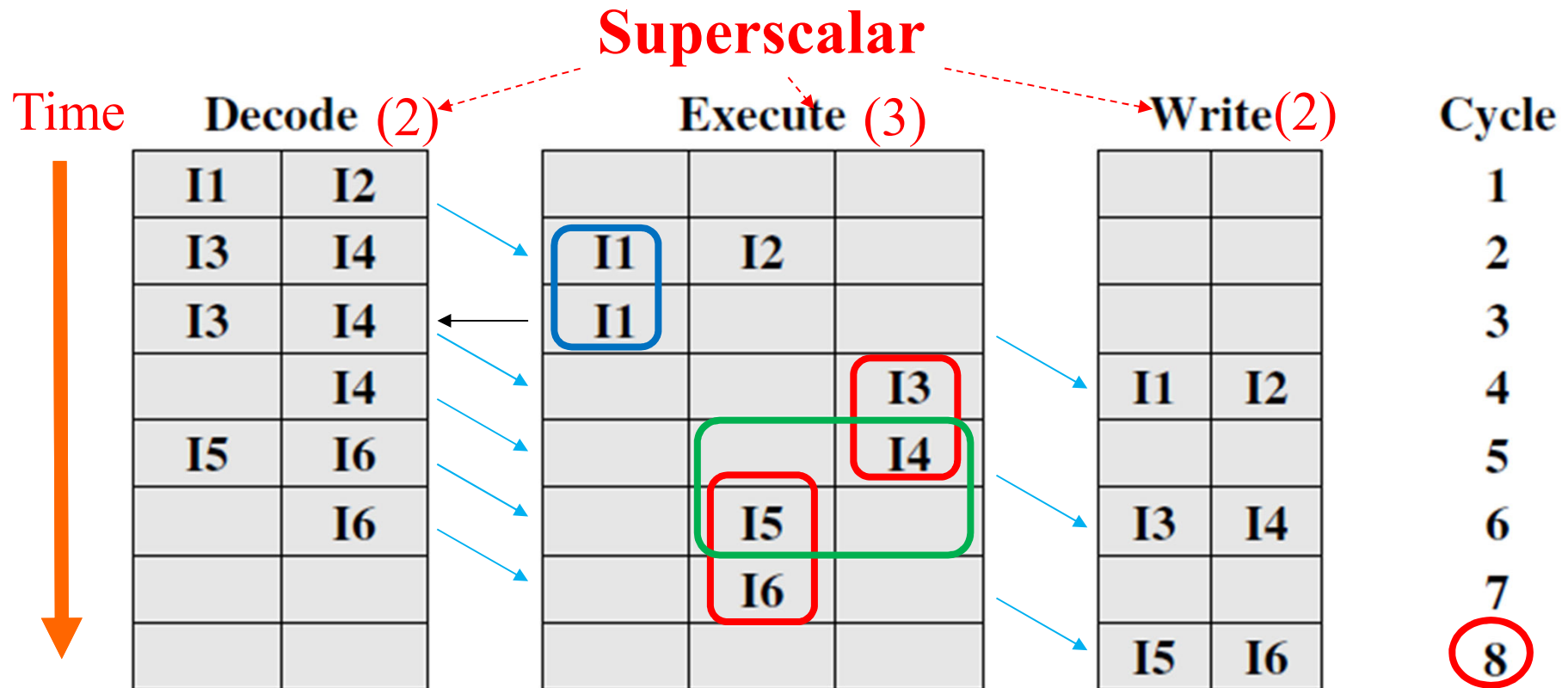
I5, I4 : true data dependency

I5, I6 : resource conflict

5 cycles -> 8 cycles

In-Order Issue

In-Order Completion (Diagram)



- Instruction issuing is stalled by
 - resource conflict
 - true data dependency
 - procedural dependency
 - function unit requiring more than one cycle.

In-Order Issue

Out-of-Order Completion

- **Out-of-Order Completion**

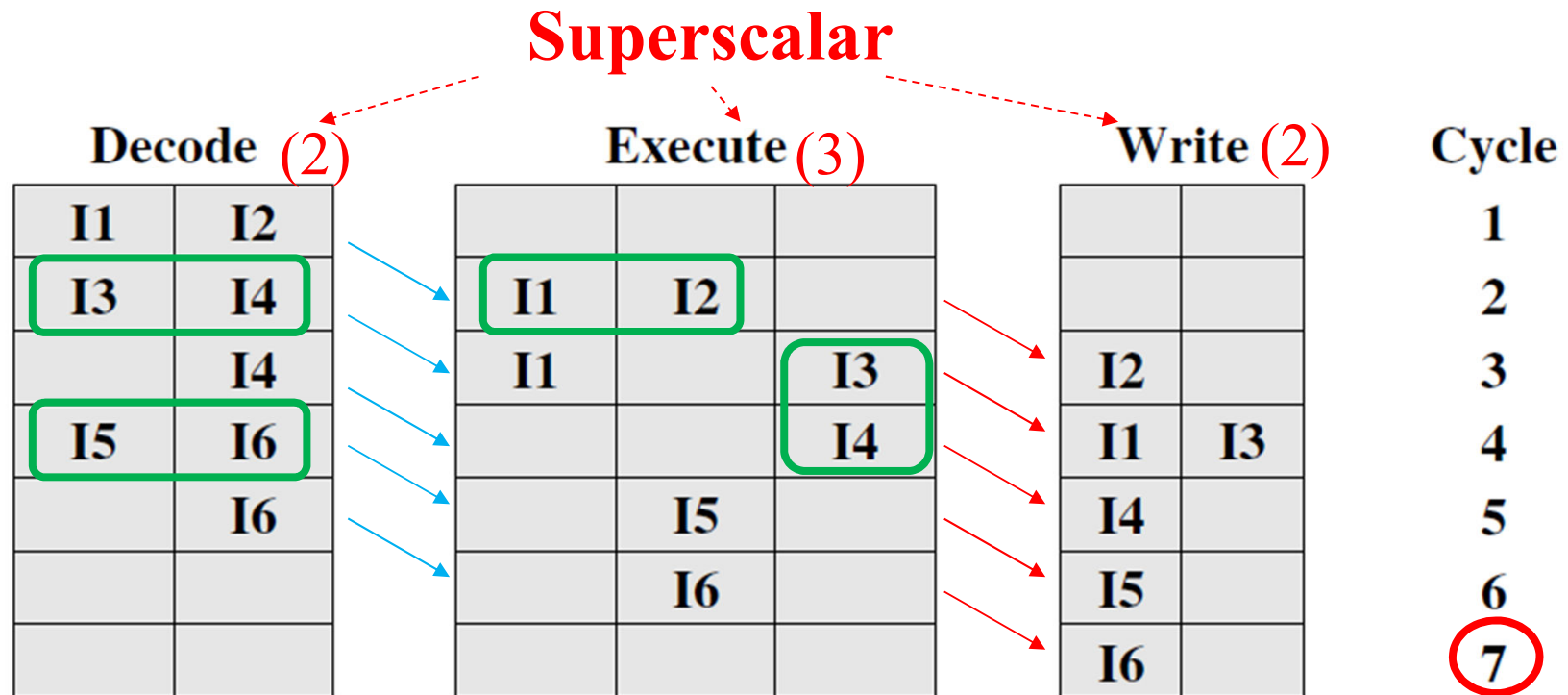
- Any number of instructions may be in the execution stage at any one time **up to the maximum degree** of machine parallelism across all functional units

Execution stage  **free**

- **Instruction issuing is stalled by**

- resource conflict
 - true data dependency
 - procedural dependency

In-Order Issue Out-of-Order Completion (Diagram)



- I1 costs 2 time cycles;
- I3, I4 need the same function unit;
- I5 need the outputs of I4;
- I5, I6 need the same function unit;
- *I2 can be completed before I1;*

Out-of-Order Issue

Out-of-Order Completion

- In-Order issue
 - No additional instructions are decoded until the conflict is resolved.
 - CPU cannot look ahead.

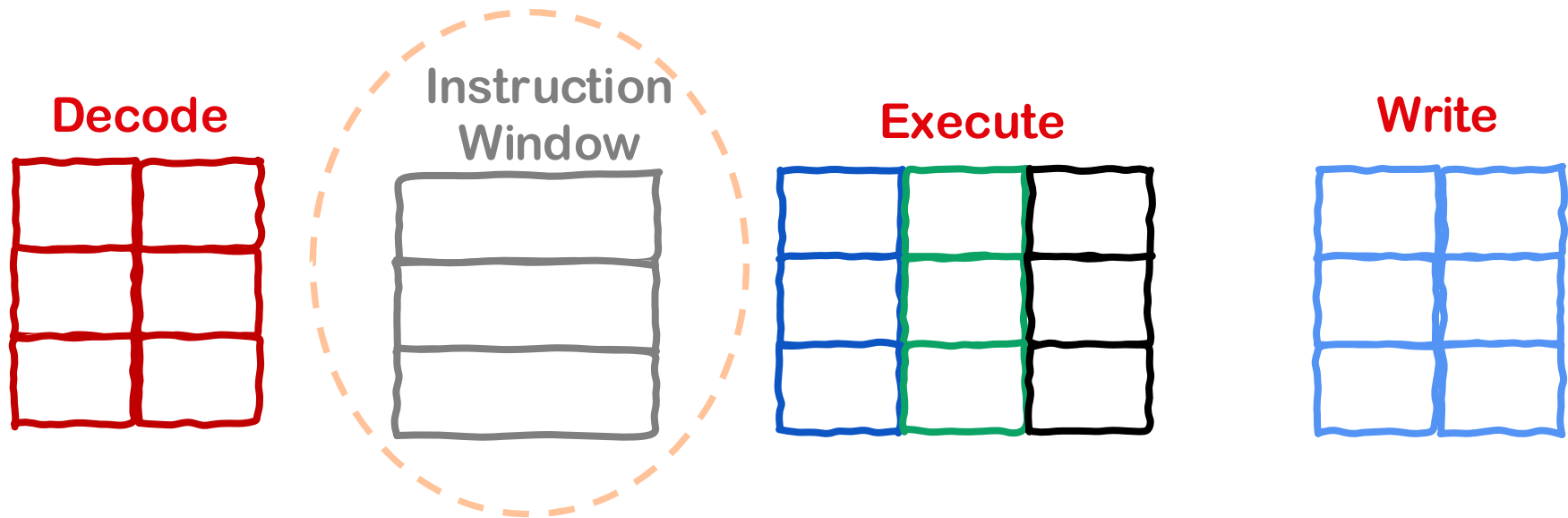
	Decode		Execute			Write		Cycle
	I1	I2						1
2.	I3	I4	1. I1	I2				2
	I3	I4	I1					3
		I4			I3	I1	I2	4
4.	I5	I6			I4			5
		I6		I5		I3	I4	6
				I6				7
						I5	I6	8

(a) In-order issue and in-order completion

Out-of-Order Issue

Out-of-Order Completion

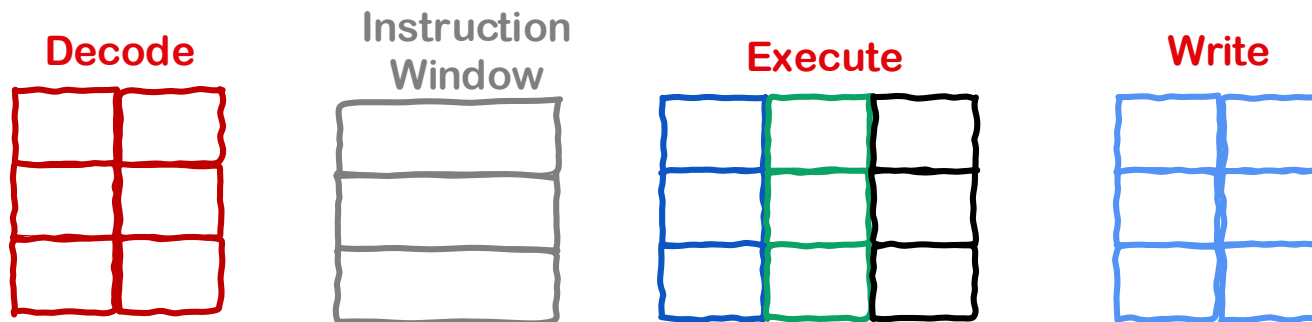
- Out-Of-Order issue
 - Decouple *decode* pipeline from *execution* pipeline
 - ***Instruction window*** is used



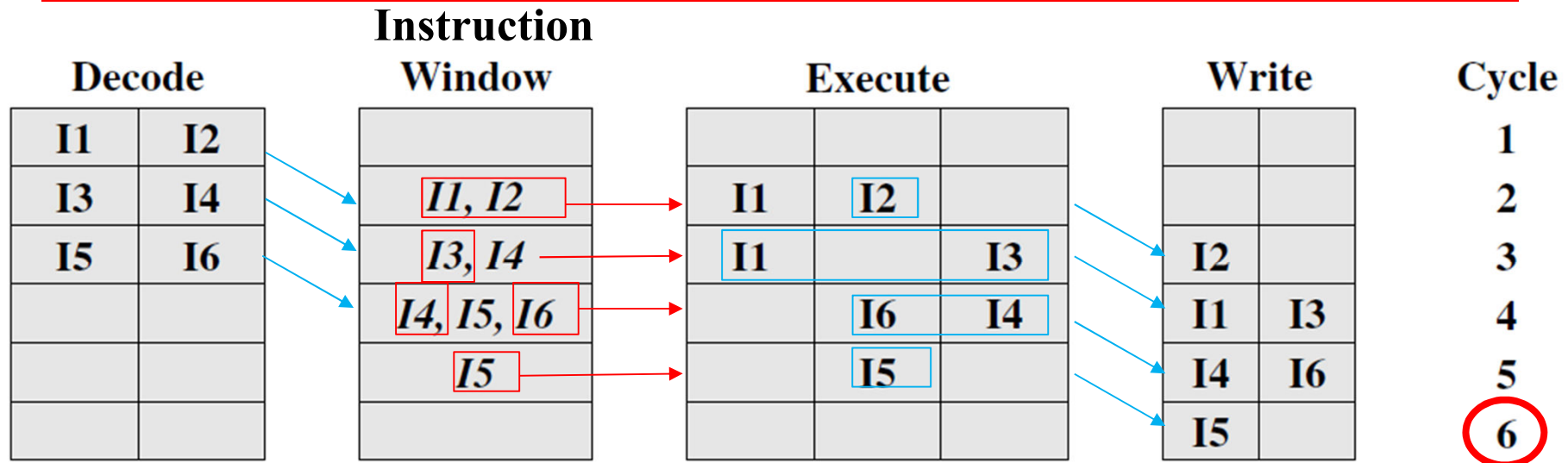
Out-of-Order Issue

Out-of-Order Completion

- ***Instruction window***
 - Can *continue to fetch and decode* until this buffer is full
 - When a functional unit becomes available, an instruction (no conflict or dependency) can be executed
 - Not an additional pipeline stage
- Processor can look ahead : it can identify independent instructions from the window.
- *The outcome must be right!*



Out-of-Order Issue Out-of-Order Completion (Diagram)



I1 costs 2 time cycles;
 I3, I4 need the same function unit;
 I5 need the outputs of I4;
 I5, I6 need the same function unit;

Instruction issuing is stalled by
 1)resource conflict
 2>true data dependency
 3)procedural dependency

The difference is that more instructions are available for issuing, reducing the probability that a pipeline stage will have to stall.

Output dependency & Antidependency

- I1: $R3 \leftarrow R3 \text{ op } R5$
- I2: $R4 \leftarrow R3 + 1$
- I3: $R3 \leftarrow R5 + 1$
- I4: $R7 \leftarrow R3 \text{ op } R4$

- **Output dependency**

- If I3 completes before I1, the result from I1 will be wrong - output (**write-after-write**) dependency

- **Write-After-Read dependency (Antidependency)**

- I3 can not complete before I2 starts as I2 needs a value in R3 and I3 changes R3

- Antidependencies and output dependencies **differ from** true data dependencies and resource conflicts. They are both examples of **storage conflicts**.

Register Renaming

- *Output and antidependencies* occur because *register contents* may not reflect the correct ordering from the program
- May result in a pipeline stall
- *Registers optimization* by the compiler(in ch15) ->making conflict more acute
- Register Renaming
 - one method to cope with storage conflicts
:duplication of resources
 - in essence, registers are allocated dynamically by the processor **hardware**

Register Renaming example

- $R3_b := R3_a + R5_a$ (I1)
- $R4_b := R3_b + 1$ (I2)
- $R3_c := R5_a + 1$ (I3)
- $R7_b := R3_c + R4_b$ (I4)
- Without subscript refers to *logical register* in instruction
- With subscript is *hardware register* allocated
- *Reduce the unnecessary output/anti-dependencies*

Register Renaming example

- Register Renaming: *Reduce the unnecessary output/anti-dependencies*

I1: R3 $\leftarrow$ R3 op R5 I2: R4 $\leftarrow$ R3 + 1 I3: R3 $\leftarrow$ R5 + 1 I4: R7 $\leftarrow$ R3 op R4		I1: R3_b $\leftarrow$ R3_a op R5_a I2: R4_b $\leftarrow$ R3_b + 1 I3: R3_c $\leftarrow$ R5_a + 1 I4: R7_b $\leftarrow$ R3_c op R4_b
--	--	--

True data dependency: I1 and I2, I3 and I4, I2 and I4

Write-After-Write dependency: ~~I1 and I3~~

Write-After-Read dependency: ~~I2 and I3, I1 and I3~~

Machine Parallelism evaluation

- 3 **hardware** techniques for performance enhancement:
 - *Duplication* of Resources (***ld/st/alu...***)
 - *Out of order* issue (***inst. windows***)
 - *Renaming* Registers (***duplicate reg.s***)
- Simulation Conclusions:
 - Not worthwhile to add function units without register renaming
 - Need instruction window large enough (more than 8)

Speedups of Machine Organizations Without Procedural Dependencies

1. Not worthwhile to add function units without register renaming.

2. Need instruction window large enough (more than 8)

2.

Window size (construction)

8

16

32

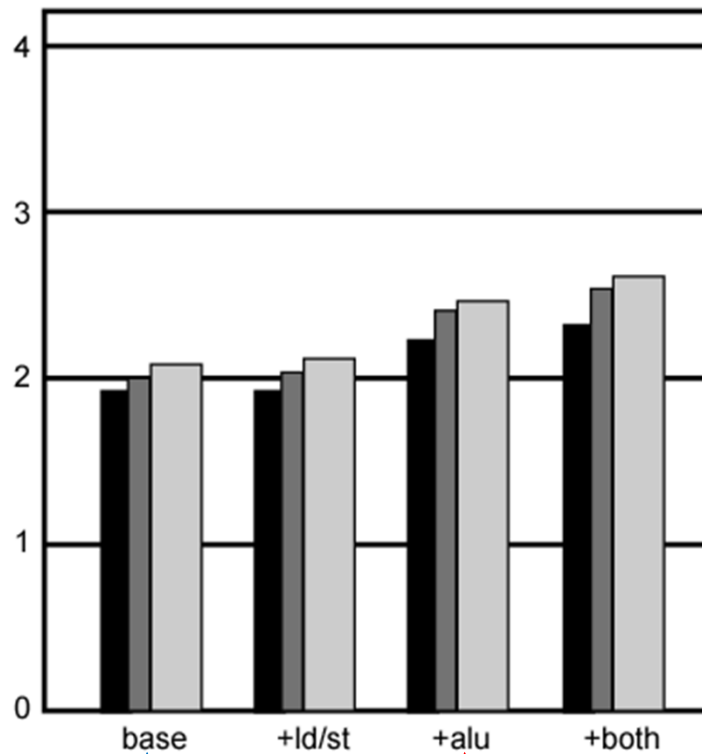
3.

With renaming

No Output/Anti-dependency.

Without renaming

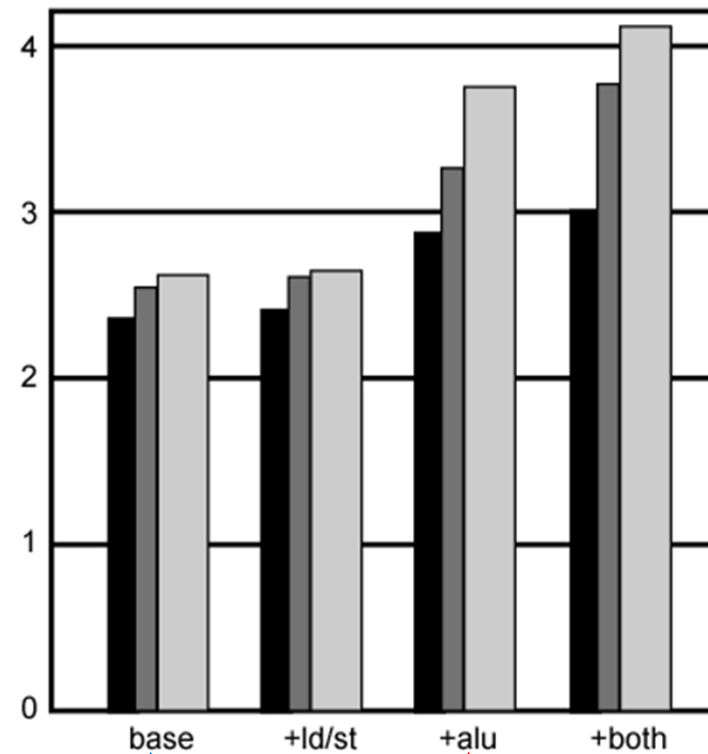
Speedup



1. None-dup

Duplication

Speedup



None-dup

Duplication

Branch Prediction

High performance pipeline machines must deal with branches

- 80486 (**CISC**) **fetches** both *next sequential instruction* after branch and *branch target instruction*
- **RISC** machines use ***delayed branch*** strategy
 - Calculate result of branch ***before*** any unusable instructions pre-fetched
 - Always execute single instruction ***immediately following*** branch
 - Keeps pipeline full while fetching new instruction stream

RISC - Delayed Branch

- Delayed branch strategy has *less appeal* for superscalar machines
 - Multiple* instructions need to execute in delay slot
 - Instruction dependence* problems
- **Branch prediction** techniques return
 - Static* branch prediction: PowerPC 601
 - Dynamic* branch prediction: PowerPC 620 and the Pentium 4

Superscalar Execution

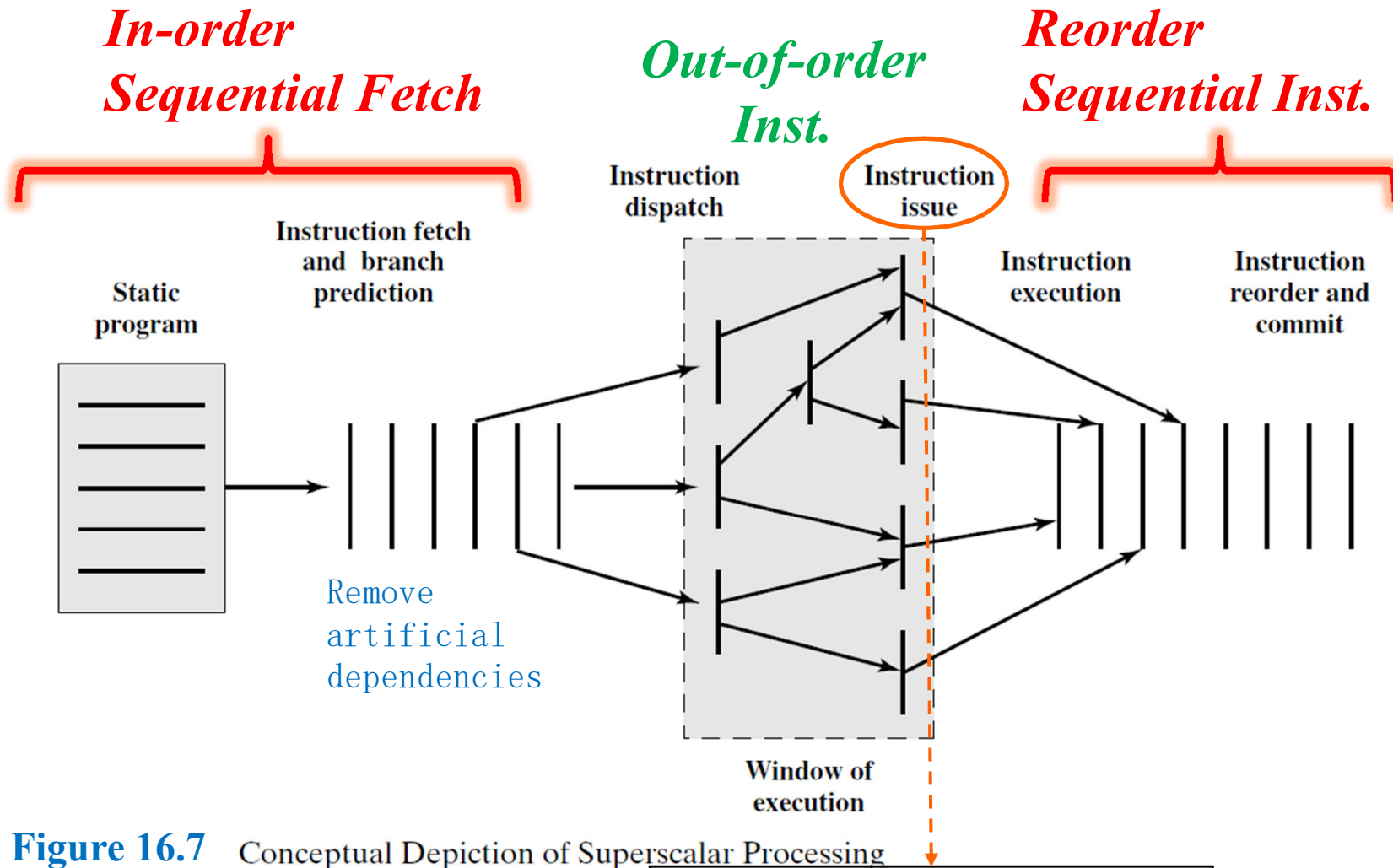


Figure 16.7 Conceptual Depiction of Superscalar Processing

*Based on Data-dependency,
Resource conflicts*

Superscalar Implementation

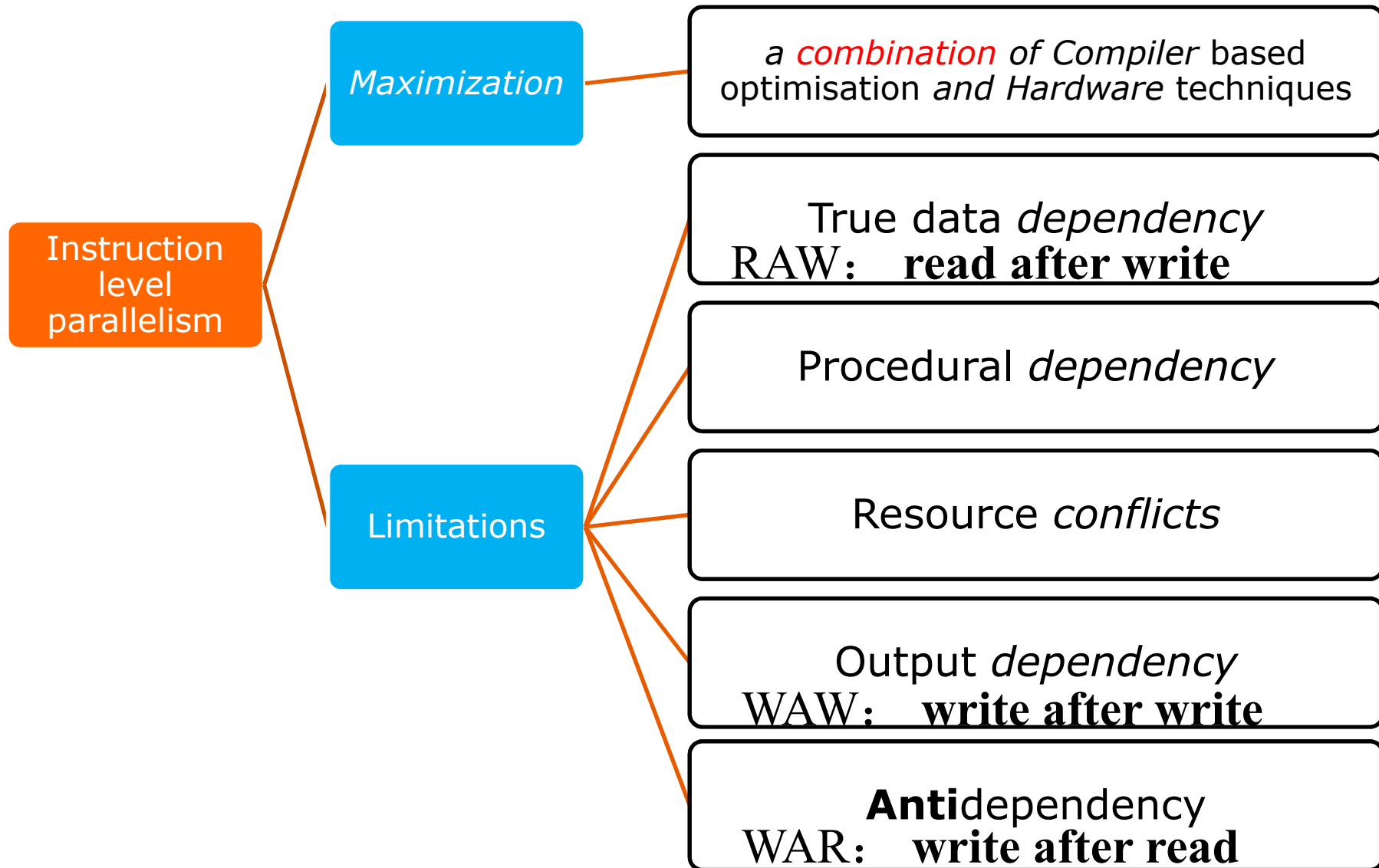
□ Key Elements:

- Simultaneously *fetch* multiple instructions
- Logic to *determine true dependencies* involving register value and mechanisms to *communicate* these values
- Mechanisms to *initiate* multiple instructions in parallel
- Resources for *parallel execution* of multiple instructions
- Mechanisms for *committing* process state in correct order

Summary

- Superscalar **vs** Superpipeline
- Instruction level parallelism(ILP) **vs** Machine Parallelism
- Five limitations of ILP: WAR,WAW,RAW, procedural dependency, resource conflicts
- How to **identify** ILP and **orchestrate** F/D/E stages in parallel?(3 instruction issue policy)
- How to eliminate some unnecessary dependencies? (Register Renaming)

Summary



Summary

The **Degree** of
Instruction
level
parallelism

True data *dependencies*

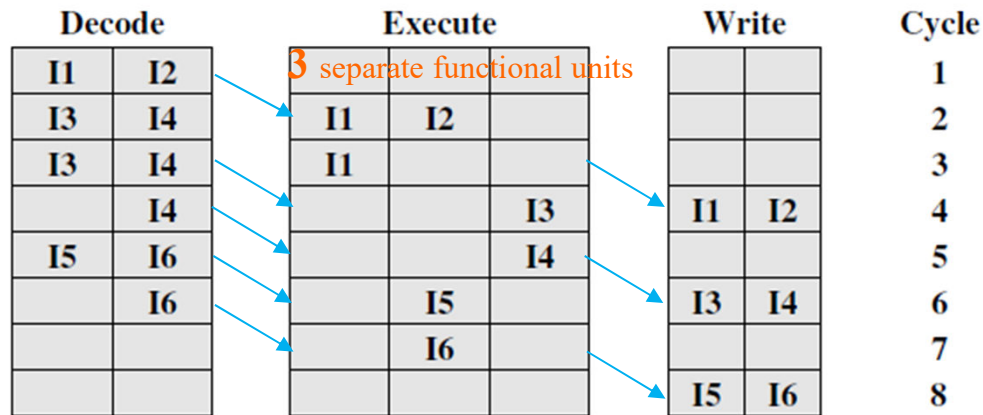
Procedural *dependencies*

operation *latency*

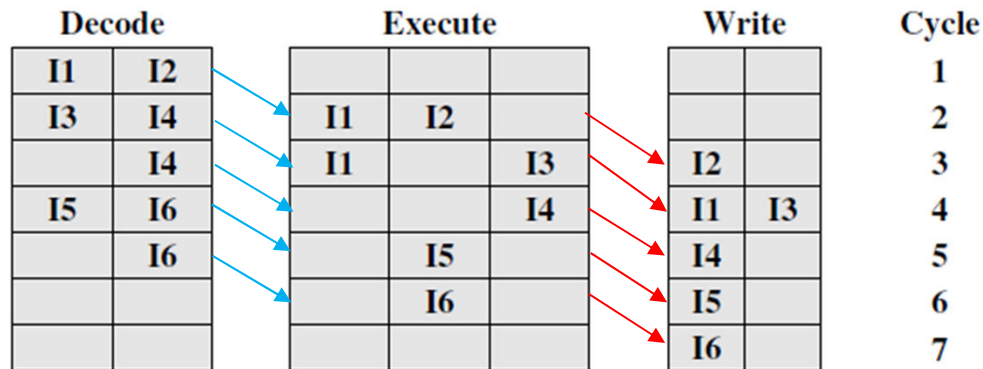
**Machine
Parallelism:**
machine *ability*
to take
advantage of
instruction level
parallelism

the ***speed*** and sophistication of the mechanisms
to find independent instructions

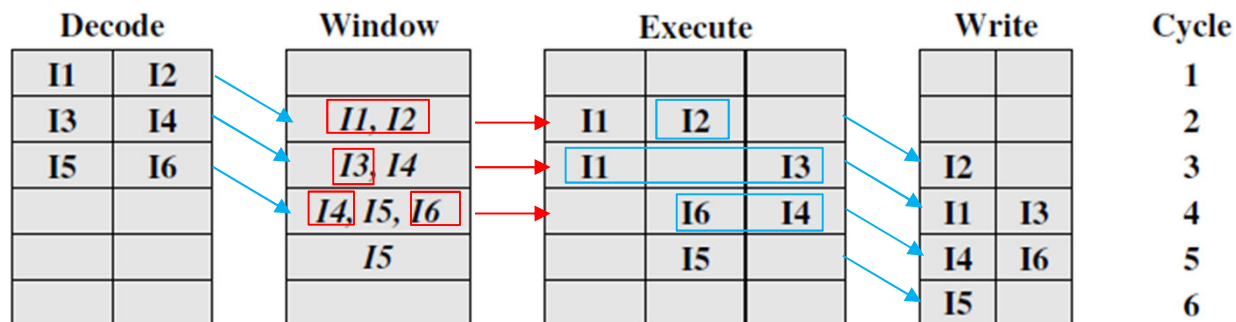
number of parallel pipelines to be fetched and
executed



(a) In-order issue and in-order completion



(b) In-order issue and out-of-order completion

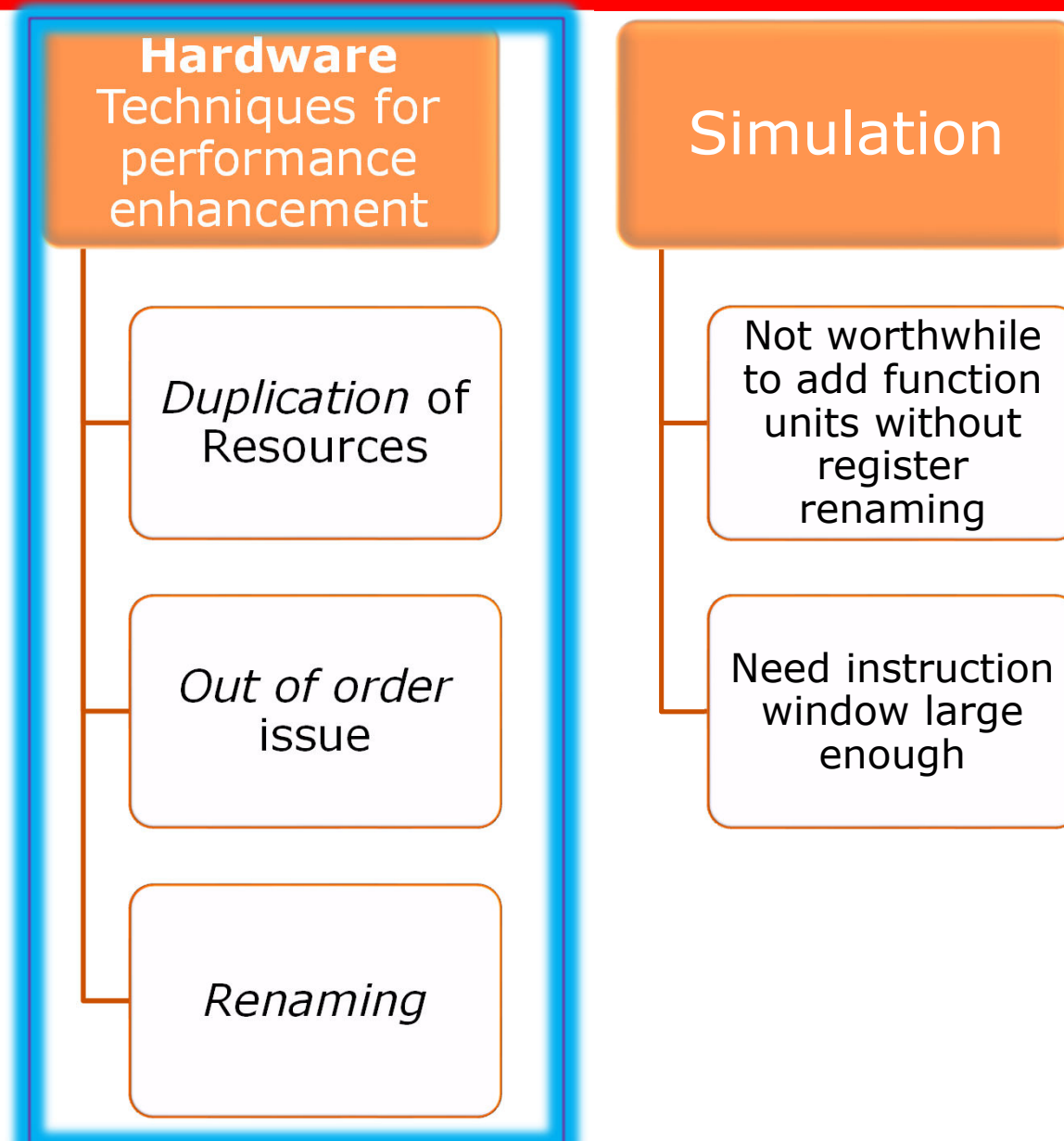


(c) Out-of-order issue and out-of-order completion

- I1 needs 2 time cycles;
- I3, I4 need the same function unit;
- I5 need the outputs of I4;
- I5, I6 need the same function unit;

Instruction window is used to **decouple** the decode and execute stages, and it is **not** an additional pipeline stage.

Summary



Homework

- Reading : Stallings ch16
- Key terms
- Review questions: 4, 7
- Problem: 3,4,5

16.5 Consider the “in-order-issue/in-order-completion” execution sequence shown in Figure 16.14.

- Identify the most likely reason why I2 could not enter the execute stage until the fourth cycle. Will “in-order issue/out-of-order completion” or “out-of-order issue/out-of-order completion” fix this? If so, which?
- Identify the reason why I6 could not enter the write stage until the ninth cycle. Will “in-order issue/out-of-order completion” or “out-of-order issue/out-of-order completion” fix this? If so, which?

Decode		Execute			Write		Cycle
I1	I2						1
	I2			I1			2
	I2			I1			3
I3	I4		I2				4
I5	I6		I4	I3	I1	I2	5
I5	I6			I3			6
		I5	I6		I3	I4	7
		I5					8
					I5	I6	9

Figure 16.14 An In-Order Issue, In-Order-Completion Execution Sequence